StudySync AI: An Intelligent Exam Preparation Planner using Constraint Satisfaction Problem (CSP)

A PROJECT REPORT

Submitted by

NAREN KARTIC B (2116241901065)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "_____ _____" is the bonafide work of "_____ _____" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. Bhuvana S, Ph.D.,

ASSISTANT PROFESSOR(SG)

Department of Computer Science

and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

Signature of the Internal Examiner

ABSTRACT

The primary objective of StudySync AI is to assist students in organizing their academic activities by considering subject requirements, examination dates, available study hours, subject priorities, and individual scheduling preferences. The system models the study planning problem as a Constraint Satisfaction Problem (CSP), where study sessions represent variables, available time slots form their domains, and academic requirements define the constraints. Hard constraints, such as avoiding overlapping sessions and respecting available study hours, ensure schedule feasibility, while soft preferences, such as prioritizing difficult subjects and allocating sufficient revision time, help improve the usefulness of the generated plan.

The proposed system incorporates classical AI search techniques, including backtracking search and the Minimum Remaining Values (MRV) heuristic, to explore possible study schedules efficiently. During the planning process, the constraint validation mechanism identifies conflicts and eliminates assignments that violate scheduling requirements. The system follows a structured workflow consisting of understanding student inputs, defining constraints, searching for feasible assignments, and validating the resulting schedule. This approach enables the generation of organized study plans while reducing the need for repetitive manual adjustments.

StudySync AI is developed using Python and Streamlit, providing an interactive, visually appealing, and user-friendly web interface. Its modular design supports clear presentation of student inputs, scheduling constraints, the AI decision-making process, and the generated study plan. By applying classical AI methods without relying on machine learning or deep learning models, the project demonstrates the practical application of foundational AI concepts to everyday academic challenges. Ultimately, StudySync AI aims to improve study organization, promote balanced preparation across subjects, reduce scheduling conflicts, and support students in developing consistent and effective study habits.

ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for enabling us to complete this project successfully. We express our sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his dedicated efforts in providing quality education at this esteemed institution. We are deeply grateful to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her constant motivation and encouragement, which inspired us throughout the completion of this project. We also thank our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We express our sincere gratitude to our College Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement.

We are especially thankful to our project guide, Dr. Bhuvana S., Ph.D., Assistant Professor(SG), for her valuable guidance, continuous encouragement, and support throughout the project. Finally, we thank our parents and friends for their direct and indirect contributions, encouragement, and support in the successful completion of this project.

NAREN KARTIC

(2116241901065)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	
	1.2 PROBLEM STATEMENT	
	1.3 OBJECTIVES	
	1.4 SCOPE	
	1.5 SECURITY REQUIREMENTS/ SECURITY CONSIDERATIONS	
	1.6 SDG ALIGNMENT	
2	**LITERATURE REVIEW**	**6**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	
	2.2 RESEARCH GAPS	
	2.3 PROPOSED SYSTEM	
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	
3	**SYSTEM DESIGN**	**12**
	3.1 SYSTEM ARCHITECTURE	
	3.2 MODULE DESIGN	
	3.3 DATA FLOW/UML DIAGRAMS	
	3.4 THREAT MODEL AND	

ATTACK SURFACE
3.5 SECURITY CONTROLS

4 SYSTEM DESCRIPTION 18
4.1 DATA COLLECTION
4.2 DATA PREPROCESSING
4.2.1 ALGORITHMS/ METHODOLOGY
4.2.2 RESULTS AND DISCUSSION

5 CONCLUSION 29

REFERENCES 30

APPENDIX

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG Allignment of StudySync AI	5
2.2	Threat modelling and security control	17

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
3.1	SYSTEM ARCHITECTURE OF STUDYSYNC AI	11
3.2	MODULE DESIGN OF STUDYSYNC AI	12
3.3	DATA FLOW DIAGRAM	13
3.5	UML USE CASE DIAGRAM	14
3.6	UML ACTIVITY DIAGRAM	15
3.7	THREAT MODEL AND ATTACK SURFACE	16
3.8	SECURITY CONTROL FLOW	17

1. INTRODUCTION

1.1 BACKGROUND

Time management and effective study planning are important factors in achieving academic success. Students often need to prepare for multiple subjects, meet assignment deadlines, attend classes, and allocate sufficient time for revision. Creating a balanced study schedule manually can be challenging, especially when subjects differ in difficulty, examination dates, preparation requirements, and individual priorities. Poor scheduling may result in overlapping study sessions, insufficient preparation, unbalanced workloads, and increased academic stress.

Artificial Intelligence (AI) provides techniques for solving complex problems by representing knowledge, evaluating alternatives, and searching for suitable solutions. Classical AI approaches, such as Constraint Satisfaction Problems (CSPs), backtracking search, and heuristic search, can be applied to scheduling problems where several conditions must be satisfied simultaneously.

StudySync AI is an intelligent study planning system designed to generate personalized study schedules based on student inputs and academic requirements. The system represents study sessions as variables, available time slots as domains, and scheduling rules as constraints. These constraints may include examination deadlines, available study hours, subject priorities, and the prevention of overlapping sessions. Classical AI search techniques explore possible assignments and identify schedules that satisfy the defined requirements.

The system is developed using Python and Streamlit to provide an interactive and visually appealing web interface. It organizes the planning process into four major stages: understanding student requirements, defining constraints, searching for feasible assignments, and validating the generated schedule. By applying classical AI concepts, StudySync AI demonstrates how intelligent problem-solving techniques can support students in managing their academic responsibilities more effectively.

1.2 PROBLEM STATEMENT

Students frequently experience difficulties in creating effective study schedules because of multiple examinations, different subject difficulties, limited preparation time, and competing academic priorities. Traditional manual planning methods require considerable effort and may produce schedules containing overlapping sessions, inadequate revision periods, or an uneven distribution of study hours.

Furthermore, fixed study timetables may become unsuitable when examination dates, available hours, or subject priorities change. Students therefore require a flexible and systematic approach that considers their individual requirements while ensuring that scheduling rules are satisfied.

The problem addressed by StudySync AI is to develop an intelligent study planning system that accepts student-specific academic information, represents scheduling requirements as constraints, and applies classical Artificial Intelligence search techniques to generate a feasible and organized study schedule. The system aims to minimize scheduling conflicts, improve the allocation of available study time, and support effective preparation across multiple subjects without relying on machine learning or deep learning techniques.

1.3 OBJECTIVES

The primary objective of StudySync AI is to develop an intelligent and constraint-aware study planning system using classical Artificial Intelligence techniques.

The specific objectives are:

1. **Collect student requirements:** Obtain relevant information about subjects, examination dates, available study hours, and preparation priorities.
2. **Model the scheduling problem:** Represent study sessions as variables, available time slots as domains, and academic requirements as constraints using the Constraint Satisfaction Problem framework.
3. **Apply classical AI search techniques:** Use backtracking search and the Minimum Remaining Values (MRV) heuristic to explore possible study schedules efficiently.
4. **Prevent scheduling conflicts:** Validate assignments to avoid overlapping sessions and violations of available study time or other defined constraints.
5. **Support personalized planning:** Allocate study sessions according to subject priorities, examination deadlines, and preparation requirements.

1.4 SCOPE

The scope of StudySync AI is limited to intelligent academic scheduling using classical Artificial Intelligence techniques. The system focuses on generating feasible study schedules based on student-provided information and predefined scheduling rules.

The system includes the following functionalities:

- **Student input management:** Collecting subjects, examination dates, available study hours, and subject priorities.
- **Constraint representation:** Defining scheduling conditions such as time availability, session conflicts, examination deadlines, and preparation requirements.
- **AI-based schedule generation:** Applying CSP modelling, backtracking search, and the MRV heuristic to identify feasible time-slot assignments.
- **Schedule validation:** Checking the generated plan for conflicts and violations of defined constraints.
- **Personalized study organization:** Supporting the distribution of study sessions according to academic priorities and preparation needs.
- **Interactive visualization:** Presenting the planning workflow and study schedule through a Streamlit-based interface.

The project is intended primarily for students who need assistance organizing their examination preparation and daily study activities. It demonstrates the use of foundational AI algorithms in a practical application.

The current scope does not include machine learning, deep learning, neural networks, automatic integration with college timetables, or direct synchronization with external calendar services. The quality of the generated schedule depends on the accuracy of the information supplied by the student and the constraints implemented in the system. Future enhancements may include automatic schedule adjustment, reminders, progress tracking, and calendar integration.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

StudySync AI processes student-provided information, including subjects, examination dates, study preferences, and available preparation time. Although the application primarily handles academic scheduling data, appropriate security considerations are necessary to protect user information and ensure reliable operation.

The following security requirements are considered:

1. **Data privacy:** Student information should be collected only when necessary for generating the study schedule. Personal or sensitive information should not be requested unless required by a clearly defined feature.
2. **Input validation:** All user inputs, including subject names, examination dates, study hours, and priorities, should be validated to prevent invalid values and inconsistent scheduling data.
3. **Data integrity:** The system should ensure that scheduling information is processed accurately and that generated schedules satisfy the defined constraints.
4. **Secure data handling:** If schedules or student details are stored, access should be restricted appropriately. Sensitive information should not be exposed through application logs, error messages, or publicly accessible files.
5. **Session management:** If user accounts or persistent storage are introduced, suitable authentication and session-management mechanisms should be implemented to prevent unauthorized access to individual study plans.
6. **Application security:** The Streamlit application and its Python dependencies should be maintained appropriately. External inputs should not be executed as code, and unnecessary access to local files or system resources should be avoided.
7. **Reliability and error handling:** The application should handle missing inputs, invalid dates, insufficient study time, and unsatisfiable constraints gracefully. When no feasible schedule exists, it should inform the user instead of presenting an invalid plan as successful.
8. **Responsible use:** Generated schedules should be presented as planning assistance rather than guaranteed academic outcomes. Students should retain the ability to review and modify their study plans.

These considerations support the development of a reliable, privacy-conscious, and user-friendly academic planning application.

1.6 SDG ALIGNMENT

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 4 – Quality Education	Target 4.4: Increase the number of people with relevant skills for employment, decent jobs, and entrepreneurship.	StudySync AI promotes, planning time-management, and independent-learning skills by helping students organize their academic preparation effectively.
2	SDG 4 – Quality Education	Target 4.5: Eliminate disparities in education and ensure equal access to education and vocational training.	The system aims to provide accessible planning assistance to students with different academic needs andpreparation requirements.
3	SDG 4 – Quality Education	Target 4.7: Ensure learners acquire the knowledge and skills needed to promote sustainable development.	StudySync AI encourages academic responsibility self-directed learning, and consistent study habits through personalized study schedules.
4	SDG 9 – Industry, Innovation and Infrastructure	Target 9.5: Enhance scientific research and technological capabilities.	The project demonstrates the application of classical Artificial Intelligence techniques, including Constraint Satisfaction Problems and heuristic search

2. LITERATURE REVIEW

LITERATURE SURVEY

Russell and Norvig [1] discussed the fundamental principles of Artificial Intelligence, including problem-solving, search algorithms, and constraint satisfaction. These concepts provide a theoretical foundation for developing intelligent systems capable of finding solutions under predefined conditions. In study planning, these principles can be applied to represent subjects, examination dates, priorities, and available study hours as a scheduling problem. Similarly, Cruz-Chávez et al. [4] investigated the application of constraint satisfaction models to university course timetabling, demonstrating how scheduling restrictions can be represented systematically to generate feasible academic timetables.

The importance of personalized educational planning was highlighted by Nabizadeh et al. [5], who surveyed learning-path personalization and recommendation methods based on learners' goals, preferences, and individual requirements. Their research emphasizes the need to adapt educational planning to different learners rather than provide identical recommendations to everyone. In a related area, Eastman [7] investigated automated space planning using computational procedures, illustrating how planning tasks can be represented as structured decision problems. These concepts support the development of StudySync AI, which allows students to configure their subjects, priorities, difficulty levels, examination dates, and daily availability to generate a personalized study timetable.

Haralick and Elliott [3] examined techniques for improving tree-search efficiency in constraint satisfaction problems, emphasizing the importance of reducing unnecessary search and selecting suitable assignments. Mackworth [2] investigated consistency in networks of relations and established important foundations for identifying inconsistencies in constraint-based problems. Dechter [6] further explored constraint processing, including consistency algorithms and search strategies. These studies provide useful theoretical foundations for scheduling applications in which multiple requirements must be satisfied simultaneously. In StudySync AI, these concepts relate to assigning study sessions to available time slots while managing scheduling restrictions and examination deadlines.

Chen et al. [8] reviewed university course timetabling methodologies and discussed the challenges of satisfying hard constraints while considering soft preferences. Their research demonstrates the complexity of generating feasible schedules when several requirements interact. Garey and Johnson [9] examined computational complexity and NP-complete

problems, providing a theoretical basis for understanding why certain combinatorial problems become difficult as their size increases. Together, these studies highlight the importance of appropriate constraint representation and efficient scheduling strategies. StudySync AI addresses a smaller, individual-level scheduling problem by considering study-hour requirements, subject priorities, available time, and examination dates.

Research on constraint satisfaction and academic scheduling demonstrates the usefulness of systematic approaches for allocating limited resources. However, institutional timetabling primarily focuses on courses, classrooms, and academic resources, whereas individual study planning requires flexibility according to personal preparation needs. StudySync AI combines ideas from classical Artificial Intelligence, constraint-based scheduling, and personalized educational planning in a lightweight Python and Streamlit application. The system provides subject configuration, availability management, timetable generation, schedule evaluation displays, workload visualization, and CSV export without relying on machine learning or deep learning. Although the current implementation uses a two-pass assignment approach, future improvements may include recursive backtracking, enhanced constraint validation, dynamic rescheduling, and more comprehensive performance evaluation**.**

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Existing study planning systems help students organize academic activities, manage examination schedules, and track their learning progress. Common approaches include manual timetables, digital calendars, task management applications, and study planner websites. These systems allow users to enter tasks, set deadlines, allocate time, and receive reminders. However, many conventional systems depend on manual scheduling and require students to make decisions about subject priorities and time allocation independently.

From a security perspective, existing digital planning applications may use authentication, access controls, encrypted communication, and secure data storage to protect user information. However, the availability of these measures varies across applications. A study planning system should validate user inputs, protect stored academic information where applicable, avoid exposing personal data, and handle invalid or inconsistent scheduling information safely.

StudySync AI adopts a constraint-based approach to academic scheduling. It combines classical AI problem-solving methods with an interactive interface to organize student requirements, search for feasible study schedules, and validate the resulting assignments.

2.2 RESEARCH GAPS

Although existing study planning applications provide useful scheduling and task management facilities, several limitations can affect their effectiveness in handling complex academic requirements.

The major research gaps identified are:

1. **Dependence on manual planning:** Many conventional planners require students to allocate study hours and arrange subjects manually, increasing the effort required to create a balanced timetable.
2. **Limited constraint handling:** Basic scheduling tools may not systematically consider multiple conditions, such as examination deadlines, available study hours, subject priorities, and overlapping study sessions simultaneously.
3. **Insufficient intelligent search:** Conventional planners generally focus on recording schedules rather than exploring alternative assignments using AI search algorithms to identify feasible solutions.
4. **Limited adaptability:** Changes in examination dates, available preparation time, or subject priorities may require considerable manual modification when schedules are not generated through a constraint-based process.
5. **Lack of systematic validation:** Some manual planning approaches do not provide automatic checks for overlapping sessions, unavailable time slots, or violations of scheduling requirements.
6. **Limited explanation of the planning process:** Basic planning applications may not clearly demonstrate how constraints influence scheduling decisions or how a valid timetable is obtained.
7. **Privacy and input-validation concerns:** Applications that collect personal schedules or academic details require appropriate data handling, input validation, and access controls where necessary.

StudySync AI addresses these gaps by representing academic planning as a Constraint Satisfaction Problem, applying classical AI search techniques, and validating the generated schedule against predefined rules. Its objective is to reduce manual effort and improve scheduling consistency without depending on machine learning or deep learning models.

2.3 PROPOSED SYSTEM

StudySync AI is an intelligent study planning system designed to generate personalized academic schedules using classical Artificial Intelligence techniques. The proposed system considers student requirements, examination dates, subject priorities, available study hours, and scheduling constraints to identify feasible study plans.

The system uses the Constraint Satisfaction Problem framework to represent the scheduling task. Study sessions are treated as variables, available time slots are defined as domains, and academic requirements are expressed as constraints. Backtracking search explores possible assignments, while the Minimum Remaining Values heuristic helps select variables with fewer remaining options. A validation mechanism checks the generated schedule for conflicts and violations.

The application is developed using Python and Streamlit, providing an interactive web interface through which students can enter their requirements and view the planning workflow. The interface presents the process through four stages: understanding student inputs, defining constraints, searching for possible assignments, and validating the final schedule.

The proposed system focuses on producing feasible and organized study schedules while reducing the effort involved in manual timetable preparation. It also provides a practical demonstration of classical AI concepts applied to educational planning.

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The StudySync AI system follows a structured workflow to convert student requirements into a feasible study schedule.

1. Student Input Collection

The system collects information about subjects, examination dates, available study hours, subject priorities, and other scheduling preferences. These details form the initial information required for generating a study plan.

2. Constraint Definition

The collected information is converted into scheduling constraints. Examples include avoiding overlapping study sessions, respecting available study hours, considering examination deadlines, and allocating sufficient preparation time to subjects.

3. CSP Modelling

The study planning problem is represented using three major components:

- **Variables:** Study sessions that need to be scheduled.
- **Domains:** Available time slots that can be assigned to each session.
- **Constraints:** Rules that determine whether a particular assignment is valid.

4. AI-Based Search

Backtracking search explores possible assignments of study sessions to available time slots. When an assignment violates a constraint, the system rejects it and considers an alternative. The Minimum Remaining Values heuristic can be used to select the next variable with the fewest remaining valid options.

5. Schedule Validation

The system checks the generated assignments against the defined constraints. It identifies scheduling conflicts and determines whether the resulting plan satisfies the implemented requirements. If no feasible assignment is found, the system should indicate that the constraints may need adjustment.

6. Study Plan Presentation

The final schedule is presented through the Streamlit interface in an organized format. Students can review the generated plan and use the information to structure their examination preparation.

This workflow provides a systematic method for transforming academic requirements into a constraint-aware study schedule using classical AI techniques.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

The proposed system aims to improve academic scheduling through the application of classical Artificial Intelligence and constraint-based problem-solving methods.

The specific objectives are:

1. To develop an intelligent study planning application that organizes academic preparation requirements systematically.
2. To collect and process student information, including subjects, examination dates, available study hours, and priorities.

3. To represent the scheduling problem using the Constraint Satisfaction Problem framework.
4. To apply backtracking search and the Minimum Remaining Values heuristic to explore possible scheduling assignments.
5. To reduce scheduling conflicts by validating study sessions against predefined constraints.
6. To support balanced preparation by considering examination deadlines, subject priorities, and available study time.
7. To provide an interactive and visually appealing interface using Python and Streamlit.
8. To demonstrate the practical use of classical AI algorithms without employing machine learning, deep learning, or neural networks.
9. To handle invalid inputs and infeasible scheduling requirements through suitable validation and error messages.
10. To provide a flexible foundation for future improvements, such as schedule adjustment, reminders, progress tracking, and calendar integration.

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

StudySync AI offers several advantages over manual study planning and basic scheduling approaches.

1. **Reduced manual effort:** The system automates the process of assigning study sessions to available time slots, reducing the effort required to prepare a timetable.
2. **Constraint-aware scheduling:** It considers predefined academic requirements and rejects assignments that violate implemented constraints.
3. **Reduced scheduling conflicts:** Automatic validation helps identify overlapping sessions and other scheduling violations.
4. **Personalized study planning:** The system uses student-provided subjects, examination dates, priorities, and available study hours to generate relevant schedules.
5. **Efficient problem-solving:** Backtracking search explores possible assignments systematically, while the MRV heuristic can reduce unnecessary exploration by prioritizing variables with fewer available choices.

Overall, StudySync AI provides a structured and intelligent approach to academic planning. By combining constraint-based modelling, classical AI search, and schedule validation, the

proposed system aims to help students organize their preparation more effectively and maintain consistent study routines.

3. SYSTEM DESIGN

3.1 System Architecture

The system architecture describes the overall structure of StudySync AI and the interaction between its major components. The system accepts student details through a Streamlit-based user interface, processes the inputs using Python, models the study planning task as a Constraint Satisfaction Problem (CSP), and applies classical AI search techniques to generate a feasible study schedule. A validation component checks the generated schedule before presenting the result to the user.

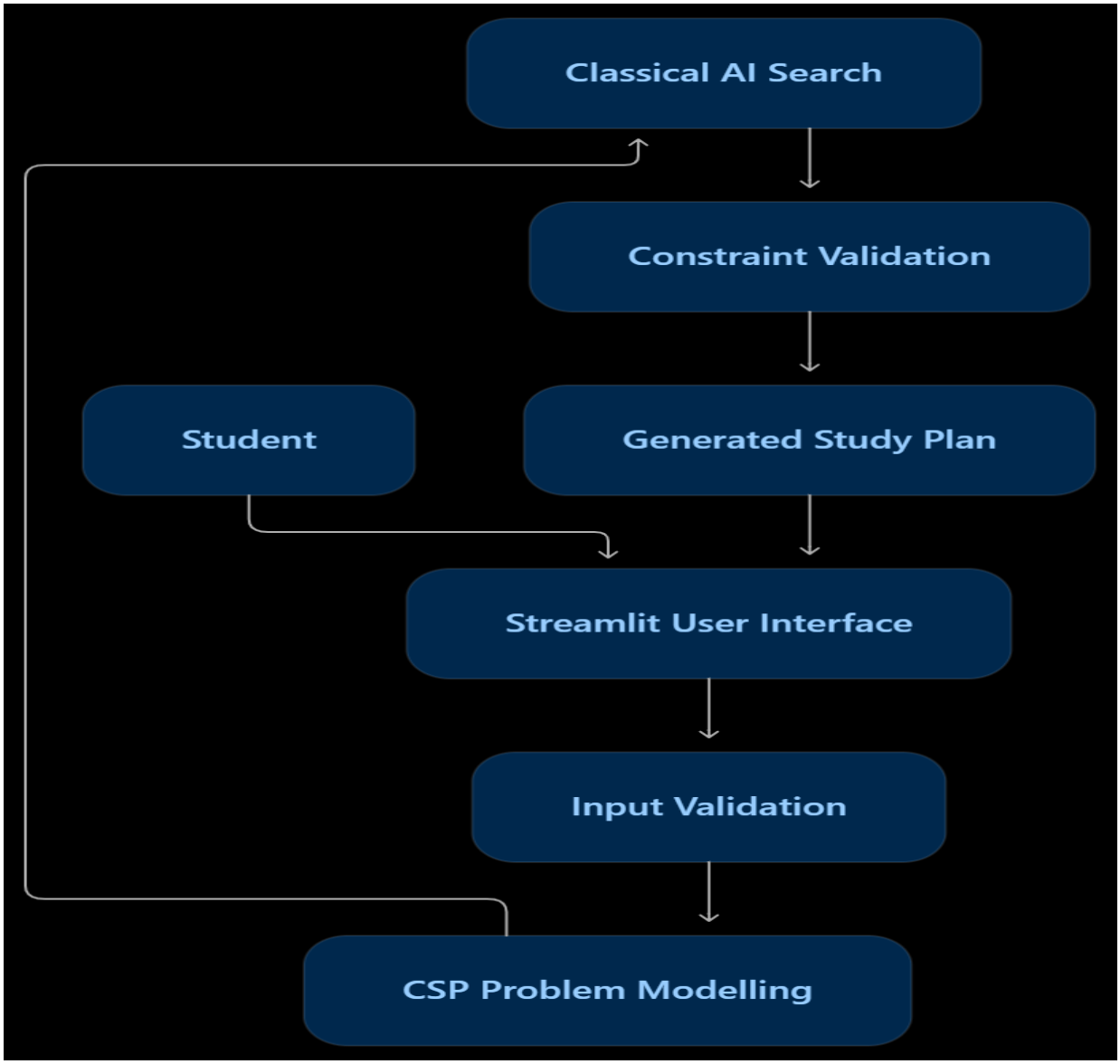


Figure 3.1: System Architecture of StudySync AI

3.2 Module Design

StudySync AI is organized into four main functional modules. Each module performs a specific task in converting student requirements into a study plan.

1. Understand Module: Collects subjects, examination dates, available study hours, and preparation requirements from the student.
2. Constrain Module: Defines scheduling rules and represents the planning problem using variables, domains, and constraints.
3. Search Module: Uses classical AI search techniques, such as backtracking and MRV, to explore possible study schedules.

4. Validate Module: Checks the resulting schedule for conflicts and violations before displaying the plan.

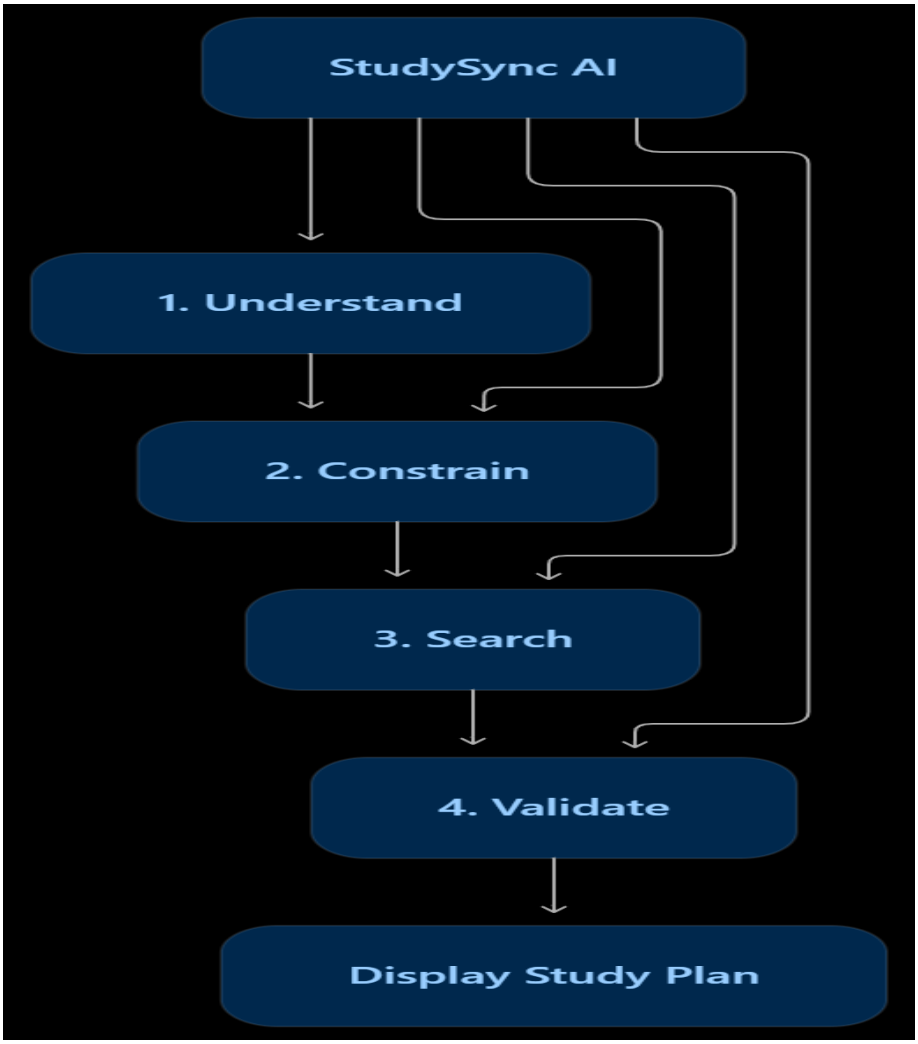


Figure 3.2: Module Design Diagram

3.3 Data Flow / UML Diagrams

Data flow diagrams illustrate how information moves through the application, while UML diagrams describe the interaction between users and system components.

A. Data Flow Diagram (DFD – Level 0)

The student provides academic details to StudySync AI. The system processes these details, generates a schedule, validates it, and returns the study plan.

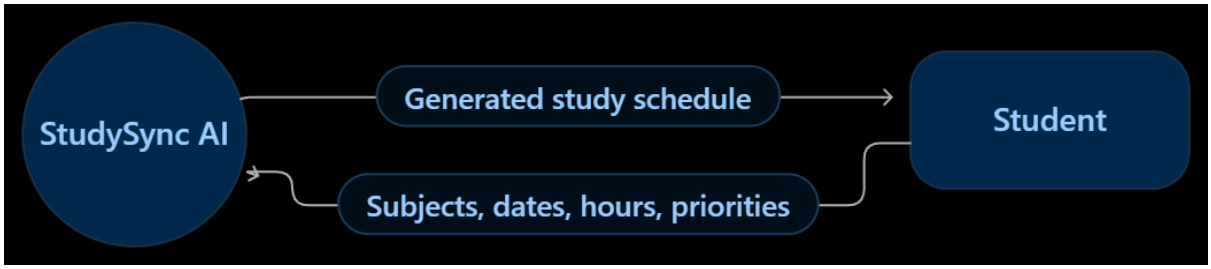


Figure 3.3 DFD – Level 0 diagram

B. Data Flow Diagram (DFD – Level 1)

This diagram expands the system into its main processing stages.

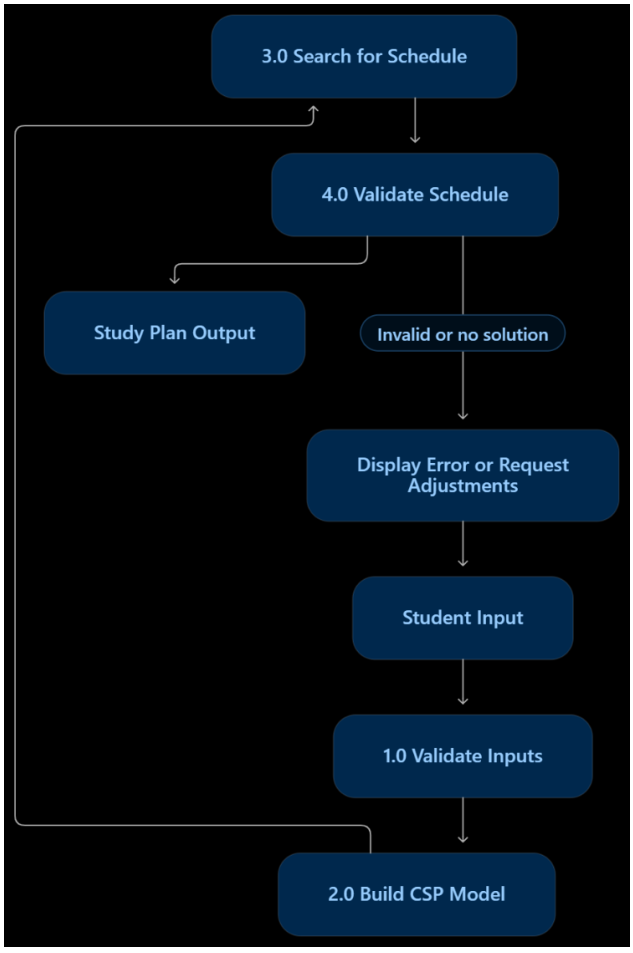


Figure 3.4 DFD Level – 1 Diagram

C. UML Use Case Diagram

The use case diagram identifies the actions performed by the student when interacting with StudySync AI.

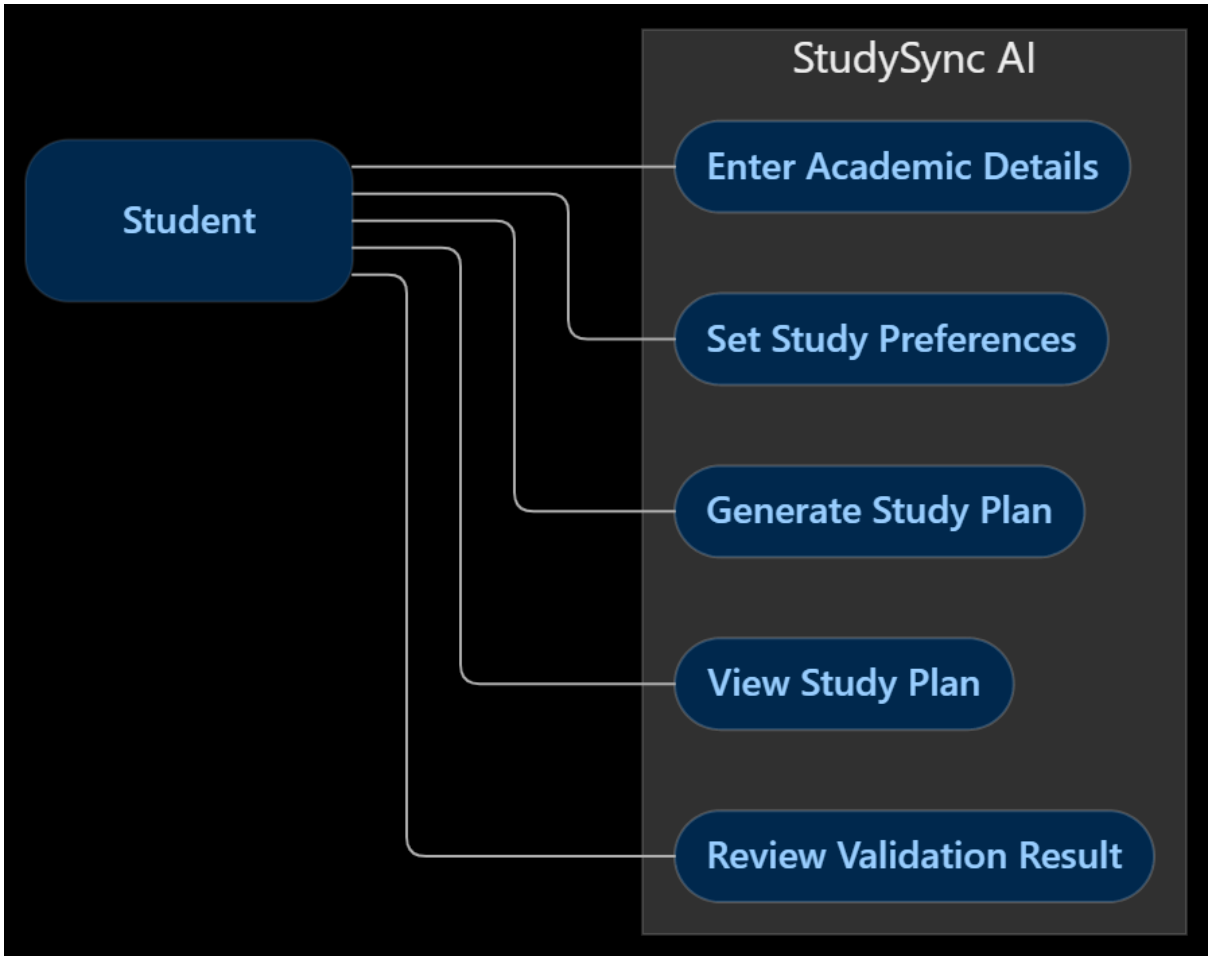


Figure 3.5 UML Use Case Diagram

D. UML Activity Diagram

The activity diagram represents the sequence of actions followed when generating a study plan.

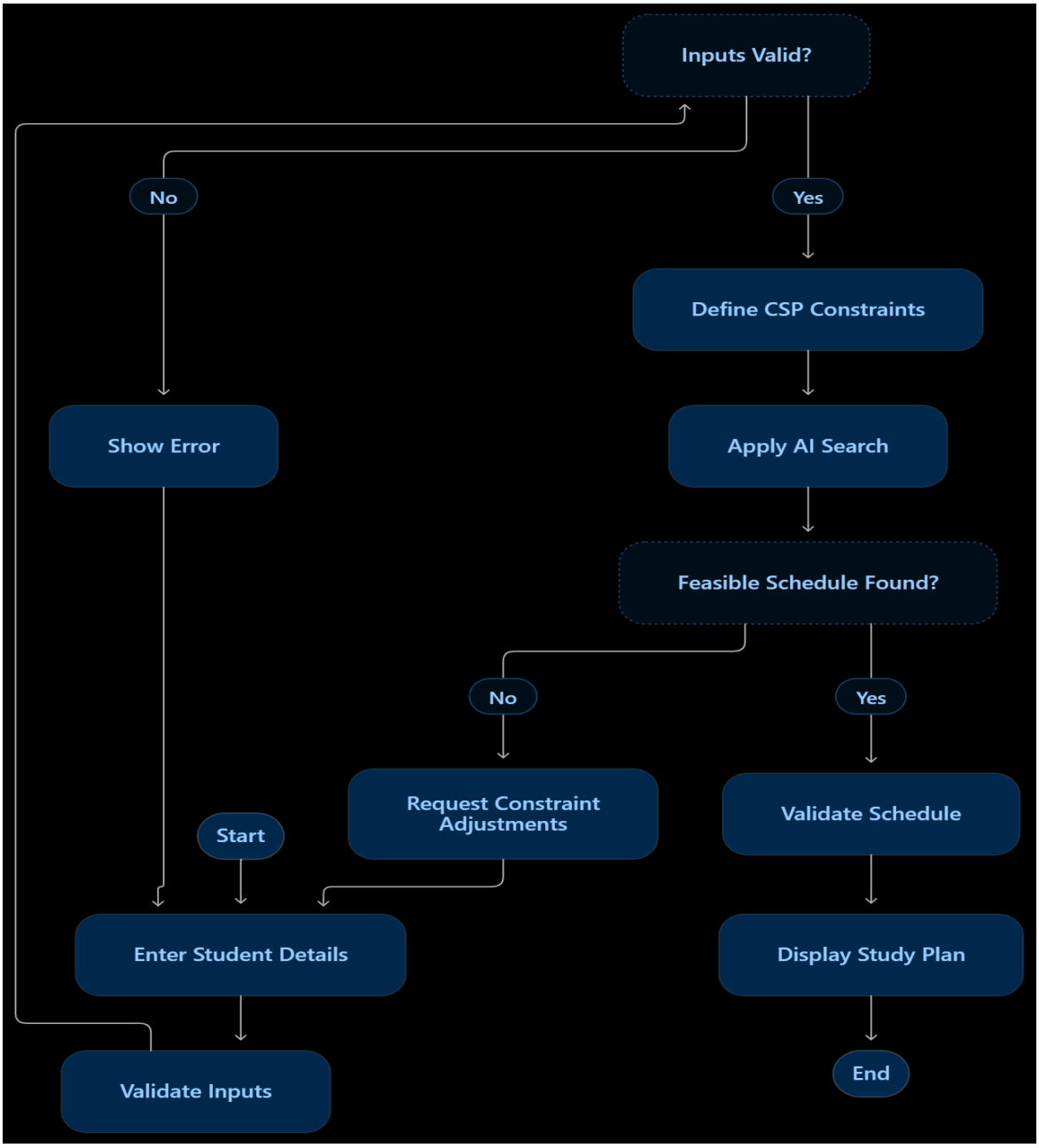


Figure 3.6 UML Activity Diagram

3.4 Threat Model and Attack Surface

The threat model identifies possible security risks associated with StudySync AI and the components that could be exposed to attacks. The primary attack surfaces include the student input fields, web interface, Python application logic, and any storage used for saving study plans. Potential threats include invalid input, injection attempts, unauthorized access to stored information, and exposure of academic data. Input validation, secure data handling, and appropriate access restrictions can help reduce these risks.

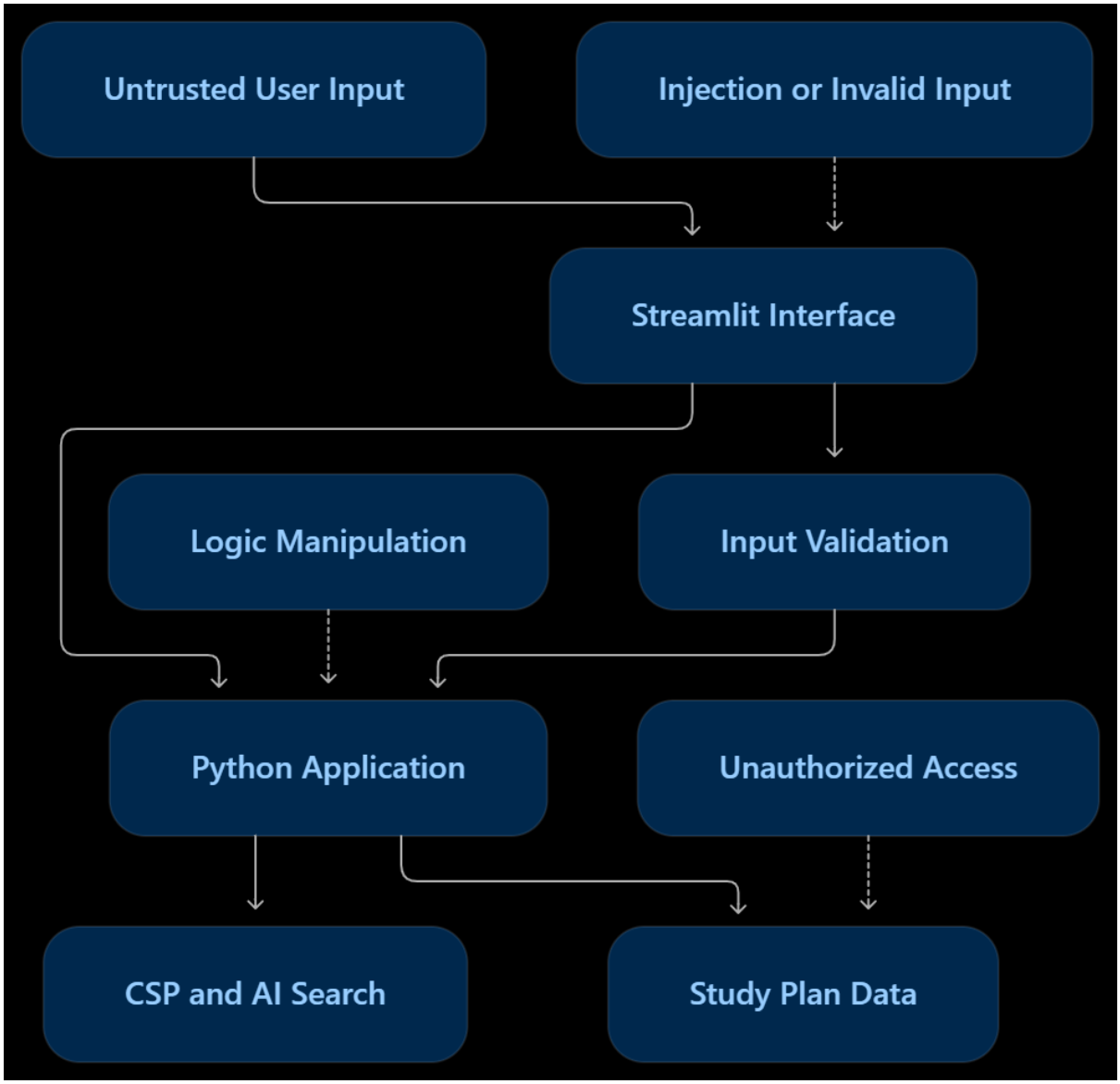


Figure 3.7 Threat Model and Attack Surface

3.5 Security Controls

Security controls are measures intended to protect the application, preserve data integrity, and ensure reliable operation. StudySync AI should validate student inputs, handle errors safely, avoid executing user-provided content as code, and protect academic information if it is stored. Access control and authentication should be added if the application supports individual accounts or shared deployment.

The following security controls are relevant to the project:

Security control	Purpose
Input validation	Reject invalid subjects, dates, study hours, and priorities.
Constraint validation	Prevent schedules that violate implemented academic rules.
Error handling	Display appropriate messages without exposing internal details.
Data privacy	Collect only necessary student information and protect stored data.
Access control	Restrict access to personal study plans when accounts or shared storage are used.
Secure dependencies	Keep Python and Streamlit dependencies updated.

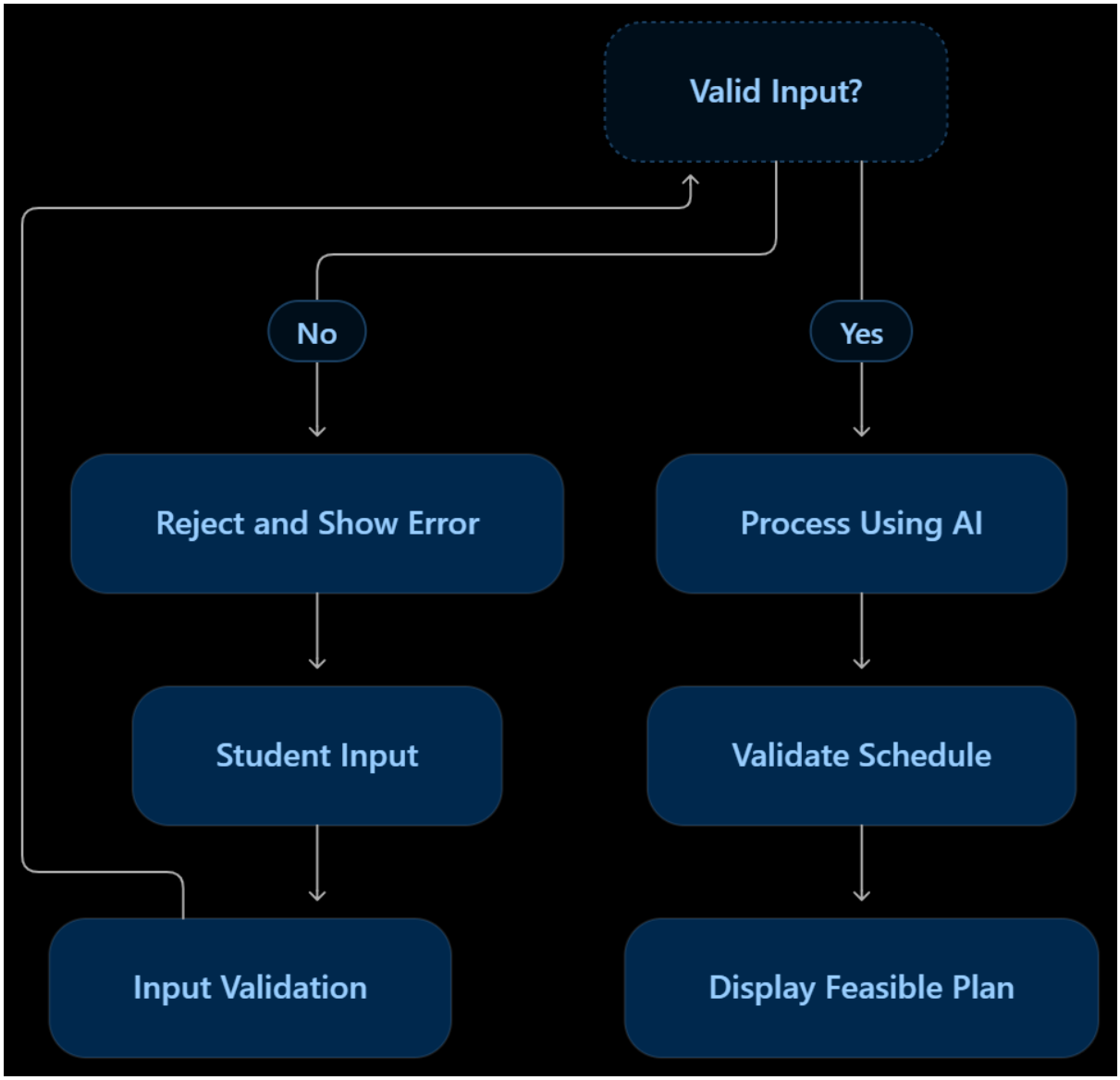


Figure 3.8 Security Control Flow

4. SYSTEM DESCRIPTION

4.1 DATA COLLECTION

Data collection is the initial stage of the StudySync AI system, in which the information required to generate a personalized study schedule is obtained from the student. The system collects academic and scheduling details through its interactive Streamlit user interface. The collected information serves as the input for the Constraint Satisfaction Problem (CSP) model and the subsequent AI search process.

The primary data collected includes subject names, examination dates, available study hours, subject priorities, and preparation requirements. Subject names identify the academic areas that require attention, while examination dates help determine preparation urgency. Available study hours define the time that can be allocated to study sessions, and subject priorities help determine how preparation time should be distributed.

The system relies on user-provided information rather than an external dataset or a machine learning training dataset. This approach is suitable because study planning is a constraint-based problem that depends on individual academic requirements. The collected information is converted into a structured representation that can be processed by the scheduling algorithm.

The following information may be collected:

- **Subject details:** Names of the subjects requiring preparation.
- **Examination information:** Examination dates and preparation deadlines.
- **Time availability:** Daily or session-based study hours available to the student.
- **Subject priorities:** Relative importance or urgency of each subject.
- **Study requirements:** Estimated preparation time and other scheduling preferences.

Appropriate input validation is required to ensure that the collected information is complete, consistent, and suitable for scheduling. Invalid dates, negative study hours, missing subject names, and inconsistent time requirements should be handled before the planning process begins.

4.2 DATA PREPROCESSING

Data preprocessing prepares the collected student information for use by the AI scheduling algorithm. Since the system receives information directly from users, preprocessing focuses on input validation, normalization, constraint identification, and the structured representation of scheduling requirements rather than cleaning a large external dataset.

Initially, the system checks whether the required fields have been provided. Subject names are checked for empty values, examination dates are verified for validity, and available study hours are checked to ensure that they fall within acceptable limits. Duplicate or inconsistent entries should also be identified where necessary.

After validation, the student information is converted into a structured format suitable for CSP modelling. Each study session is represented as a variable, the available time slots form its domain, and the academic scheduling requirements are expressed as constraints.

For example, if a student has three subjects to prepare for and several available time slots, the system must assign each required study session to an appropriate slot without violating the defined rules. Constraints may include avoiding overlapping sessions, respecting available study hours, and allocating sufficient preparation time before examinations.

Preprocessing also involves identifying infeasible requirements. If the requested preparation time exceeds the available study time, the system should report the conflict or request revised inputs rather than generating an invalid schedule.

The output of preprocessing is a validated and structured set of student requirements that can be passed to the AI search algorithm.

4.2.1 ALGORITHMS / METHODOLOGY

StudySync AI uses classical Artificial Intelligence techniques to generate feasible study schedules. The principal methodology is based on the Constraint Satisfaction Problem (CSP) framework, supported by backtracking search and the Minimum Remaining Values (MRV) heuristic.

1. Constraint Satisfaction Problem (CSP)

A CSP consists of three main components:

- **Variables:** The study sessions that must be scheduled.
- **Domains:** The available time slots that can be assigned to each session.
- **Constraints:** The rules that determine whether an assignment is valid.

For example, a study session for Mathematics may be assigned to an available evening slot, provided that the slot does not overlap with another session and satisfies the relevant scheduling requirements.

2. Backtracking Search

Backtracking search is a systematic search algorithm that assigns values to variables one at a time. After each assignment, the algorithm checks whether the current partial schedule satisfies the applicable constraints. If an assignment causes a violation, the algorithm reverses that assignment and explores another possibility.

The general procedure is:

1. Initialize the CSP using the validated student inputs.
2. Select an unassigned study-session variable.
3. Choose an available time slot from its domain.
4. Check whether the assignment satisfies the constraints.
5. If valid, continue assigning the remaining variables.
6. If invalid, try another available time slot.
7. Backtrack when no valid assignment remains.
8. Return a feasible schedule if one is found; otherwise, report that no solution was found under the current constraints.

3. Minimum Remaining Values (MRV) Heuristic

The MRV heuristic selects the unassigned variable with the fewest remaining legal values in its domain. This helps the search process address the most constrained study session first.

For example, if one subject can be scheduled in only two available time slots while another can be scheduled in five, MRV selects the first subject. This can reveal conflicts earlier and reduce unnecessary search in suitable CSP instances.

4. Constraint Validation

After a schedule is generated, the system checks whether the assignments satisfy the implemented scheduling rules. These may include the prevention of overlapping sessions, adherence to available study hours, and compliance with examination-related deadlines.

METHODOLOGY / MODULE DESCRIPTION

MODULE – 01 STUDENT PROFILE

Purpose

The Understand module collects academic information from the student and prepares it for the planning process. It acts as the input layer of StudySync AI.

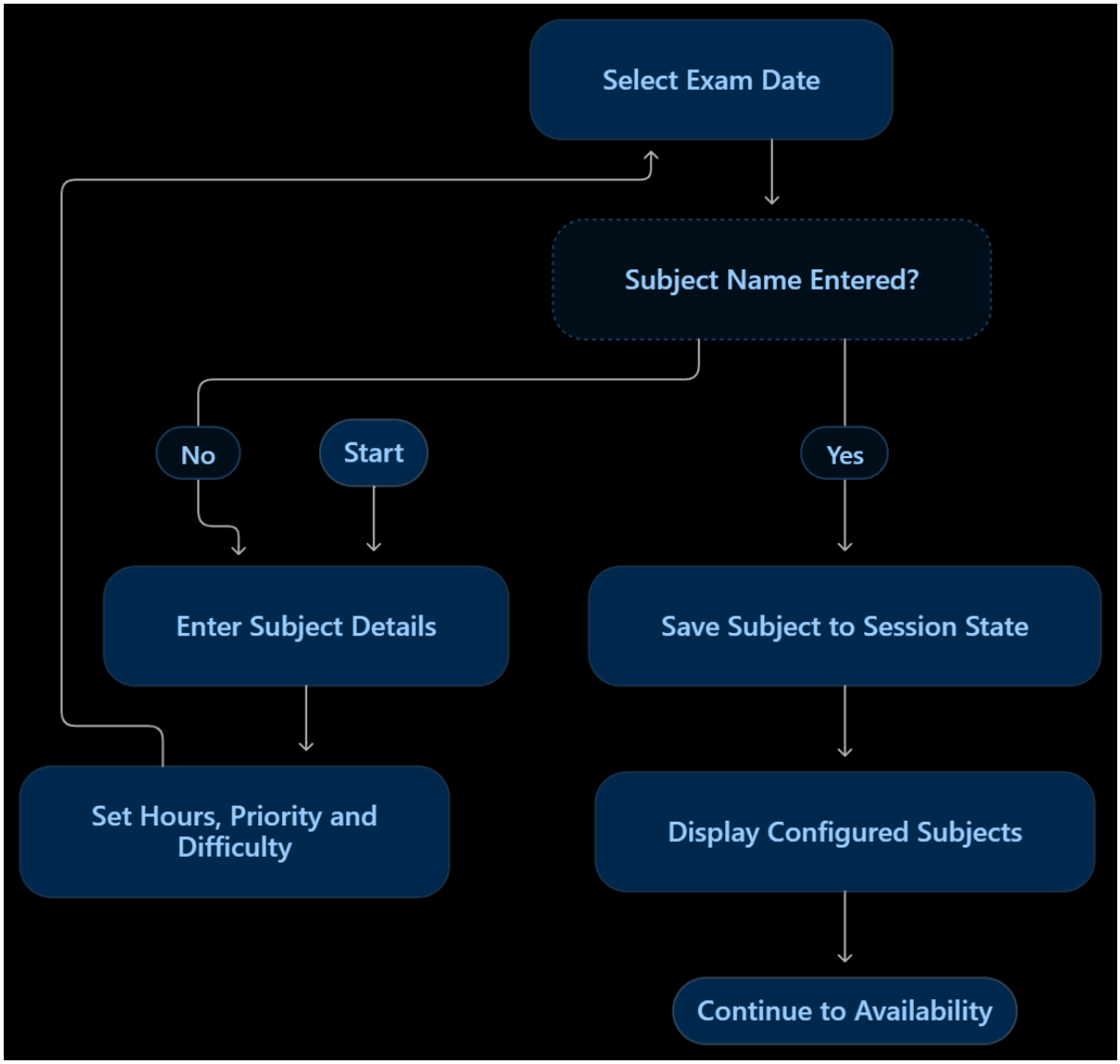
Functions

- Collect subject names and examination dates.
- Record available study hours.
- Identify subject priorities and preparation requirements.
- Collect preferred study periods and other preferences, where supported.
- Check whether required fields have been provided.

Module details

Component	Description
Input	Academic details and student preferences
Processing	Collect, organize and check input data
Output	Structured student requirements
Main objective	Ensure the planner receives usable information

Example: A student enters Web Programming, Ethical Hacking, their examination dates, and the hours available for preparation. The module organizes these details for the next stage.



MODULE – 02 AVAILABILITY

Purpose

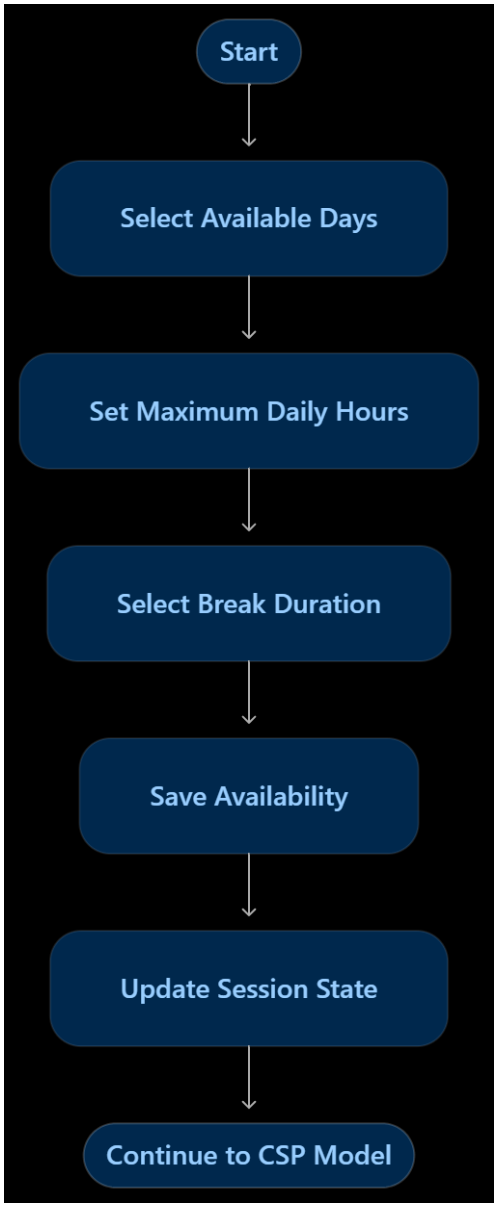
Defines when the student is available to study and the maximum number of study hours per day.

Implemented features

- Select active study days from Monday to Sunday.
- Set maximum daily study hours, from 2 to 12.
- Select a break duration.
- Save availability settings in session state.

Input	Processing	Output
Available days	Filter eligible calendar dates	Allowed study days
Maximum hours	Create hourly time slots	Available time slots
Break duration	Save selected setting	Stored preference

Implementation note: The CSP code currently uses study days and maximum daily hours, but it does not yet apply the selected break duration when generating the schedule.



MODULE – 03 CSP MODEL

Purpose

The Search module attempts to assign study sessions to time slots while satisfying the CSP constraints. It is the main problem-solving stage of StudySync AI.

Algorithms

A. Backtracking Search

Backtracking assigns a value to a variable, checks whether the partial schedule remains valid, and reverses an assignment when it causes a conflict.

B. Minimum Remaining Values (MRV)

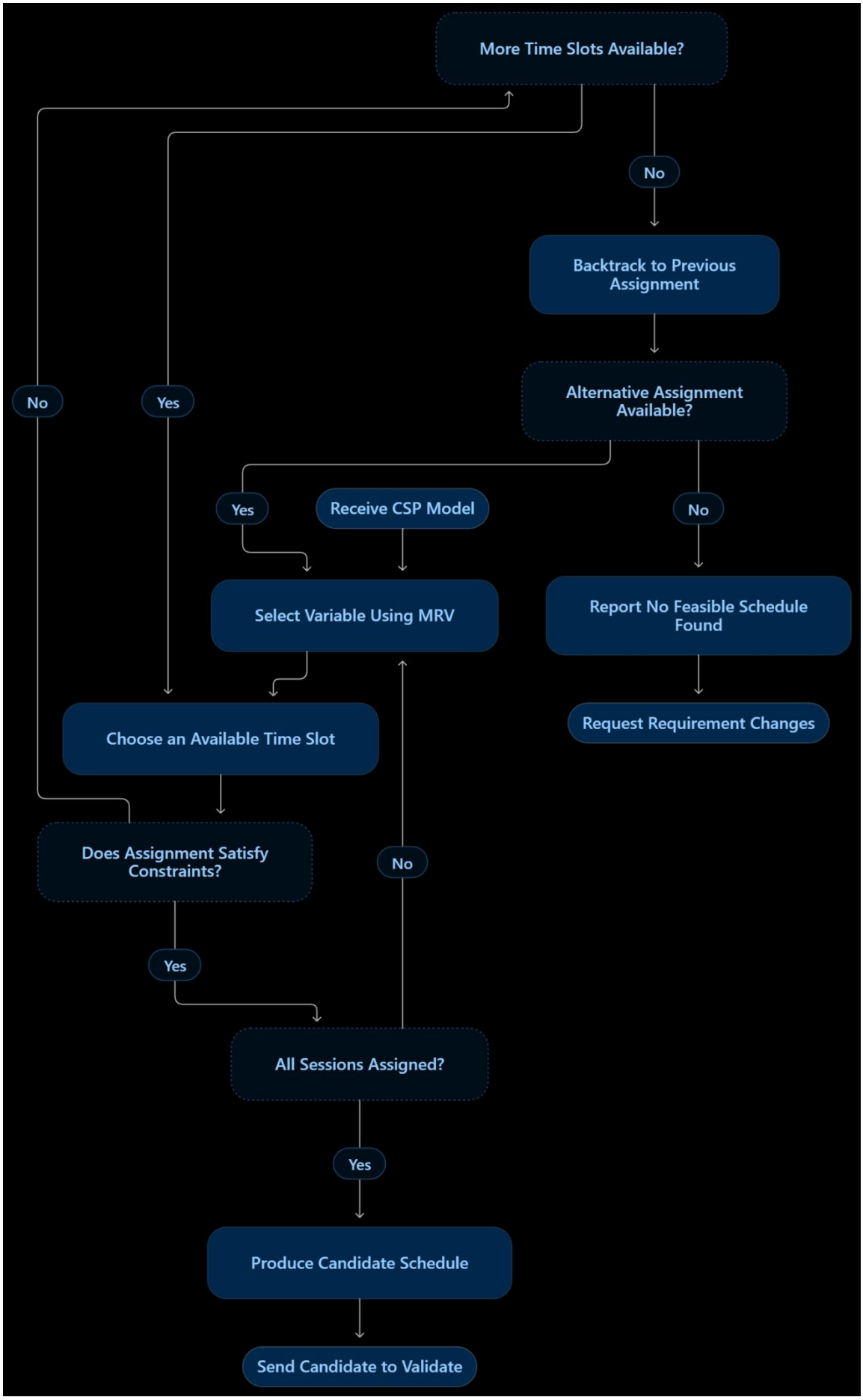
MRV selects the unassigned variable with the fewest legal values remaining. This can help expose difficult scheduling choices early.

Search process

Step	Operation
1	Receive the CSP model
2	Select an unassigned study session
3	Choose a suitable available time slot
4	Check the assignment against constraints
5	Continue if valid; backtrack if it creates a conflict
6	Return a feasible schedule or report that no solution was found within the search process

Example: If Web Programming and Ethical Hacking are both assigned to 6–7 PM, the search process must reject that conflicting assignment and try another available slot.

Technical note: MRV is a variable-selection heuristic; Backtracking is the search method. MRV alone does not generate the schedule.


More Time Slots Available?
No
Backtrack to Previous Assignment
Alternative Assignment Available?
No
Yes
Yes
Receive CSP Model
No
Select Variable Using MRV
Report No Feasible Schedule Found
Choose an Available Time Slot
Request Requirement Changes
Does Assignment Satisfy Constraints?
No
Yes
All Sessions Assigned?
Yes
Produce Candidate Schedule
Send Candidate to Validate

MODULE – 04 EVALUATION

Purpose

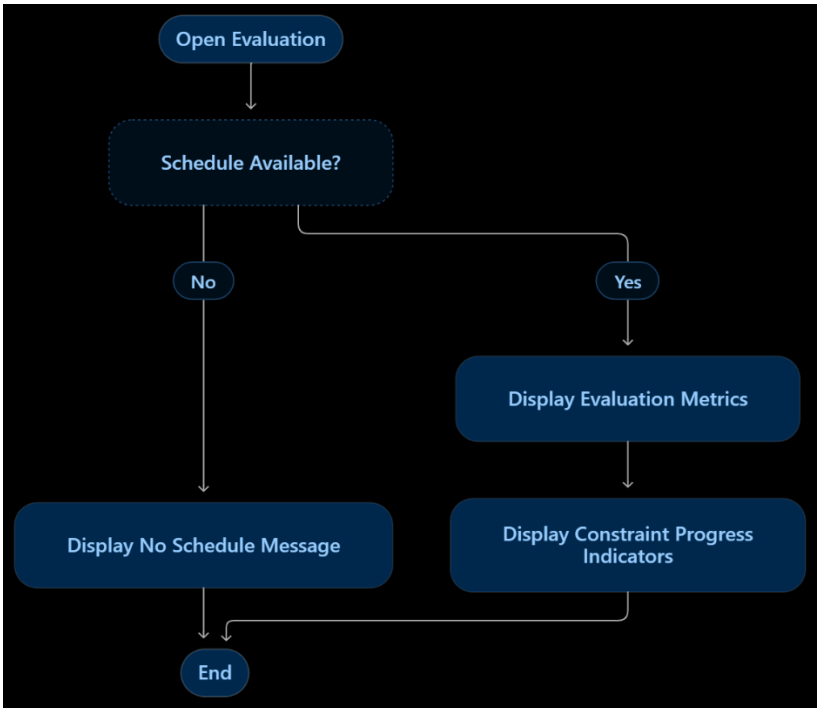
Presents evaluation metrics when a schedule exists in session state.

Implemented features

- Checks whether a schedule is present.
- Displays a constraint satisfaction rate.
- Displays backtracking steps and execution time.
- Shows progress indicators for hard and soft constraints.

Metric displayed	Current implementation
Constraint satisfaction rate	Hardcoded as 100%
Backtracking steps	Hardcoded as 14
Execution time	Hardcoded as 0.042 seconds
Soft constraint progress	Hardcoded as 88%

Important: These are currently fixed display values, not measurements calculated from the generated schedule. Before your final review, calculate these metrics from actual scheduling results or label them as illustrative values.



MODULE – 05 DASHBOARD AND ANALYTICS

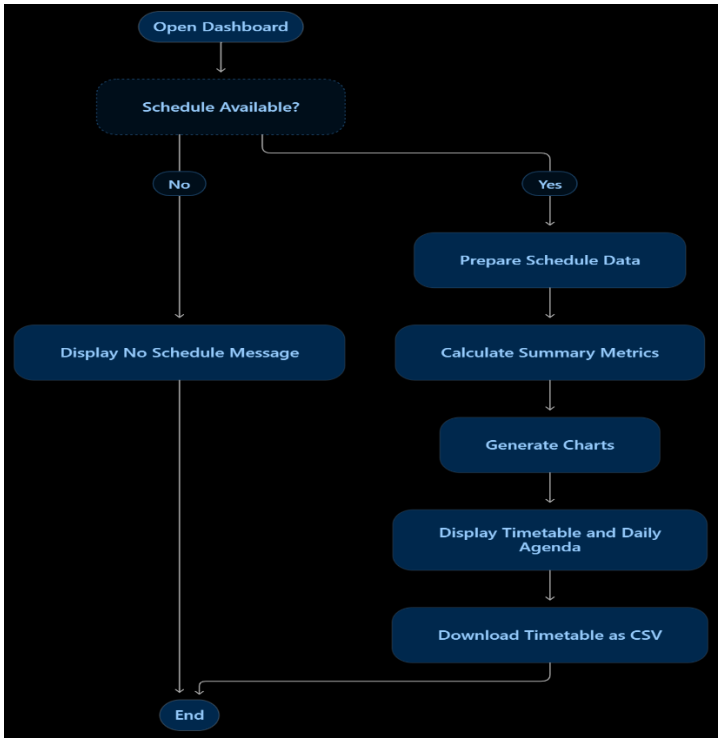
Purpose

Displays the generated schedule and summarizes study workload through tables, charts, and a downloadable timetable.

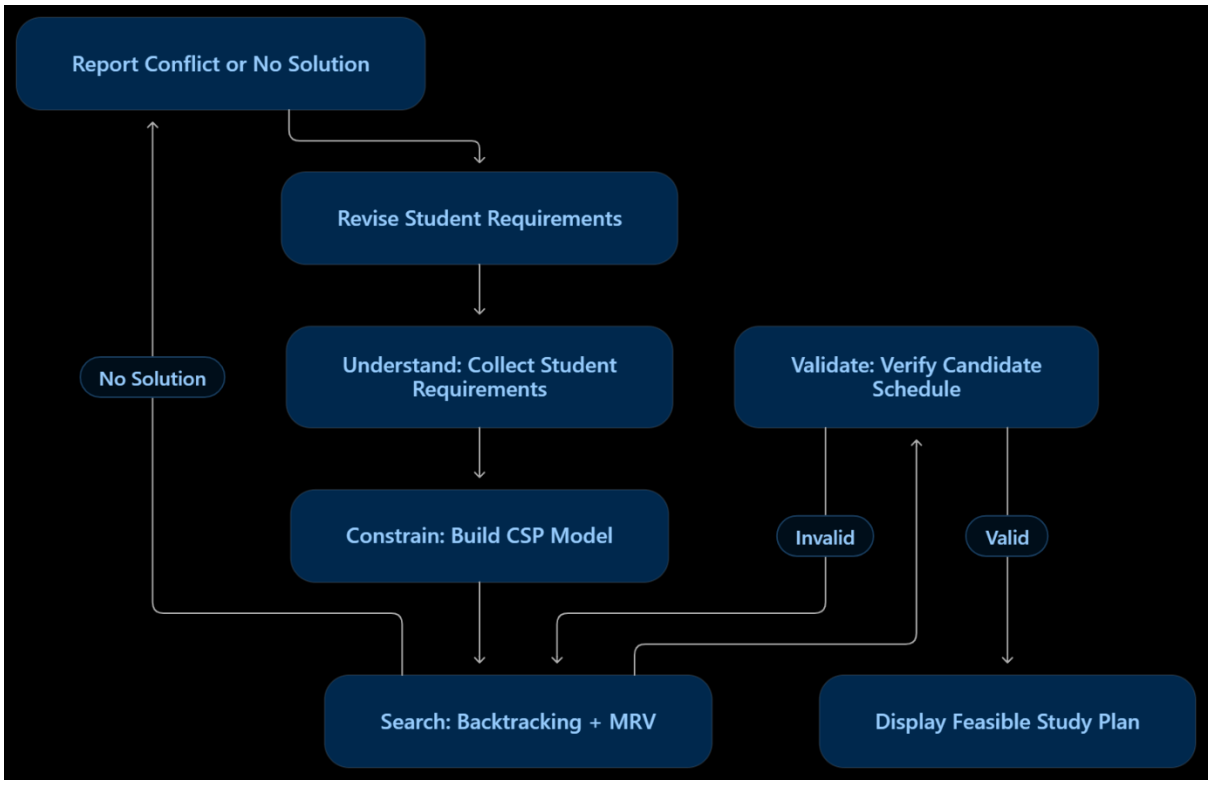
Implemented features

- Total scheduled hours and study-day span.
- Pre-exam focus and mixed-study hour summaries.
- A donut chart showing hours allocated per subject.
- A stacked bar chart showing daily workload.
- A timetable grid and daily agenda.
- CSV export of the timetable.

Input	Processing	Output
Generated schedule	Calculate subject and date summaries	KPI metrics
Scheduled sessions	Group by subject and date	Charts
Session date and time	Organize into a timetable	Master timetable
Schedule records	Convert to CSV	Downloadable file



INTERACTION BETWEEN ALL FOUR MODULES



4.2.2 RESULTS AND DISCUSSION

StudySync AI is designed to transform student-provided academic requirements into an organized study schedule through constraint-based reasoning and classical AI search. The Streamlit interface presents the planning workflow through four stages: Understand, Constrain, Search, and Validate.

During the input stage, the system obtains the information needed to describe the student's study requirements. The constraint modelling stage represents the scheduling problem in terms of variables, domains, and rules. The search stage explores possible assignments, while the validation stage checks whether the resulting schedule satisfies the implemented constraints.

The expected benefits include reduced manual scheduling effort, improved organization of study sessions, and fewer conflicts caused by overlapping assignments. The use of MRV can also improve search efficiency by selecting highly constrained variables first, although the actual improvement depends on the problem size, domain structure, and implementation.

5. CONCLUSION

StudySync AI is an intelligent study planning system designed to simplify academic scheduling through the application of classical Artificial Intelligence techniques. The project addresses the challenges associated with manual study planning, including inefficient time allocation, overlapping study sessions, difficulty prioritizing subjects, and inadequate preparation before examinations. By modelling the scheduling problem as a Constraint Satisfaction Problem (CSP), the system provides a structured approach to organizing academic requirements and identifying feasible study schedules.

The proposed system incorporates classical AI techniques, including backtracking search and the Minimum Remaining Values (MRV) heuristic, to explore possible study-session assignments while considering predefined scheduling constraints. The validation mechanism ensures that the generated schedule is checked against the implemented rules before being presented to the student. The application is developed using Python and Streamlit, providing an interactive and visually appealing interface that simplifies user interaction and presents the planning workflow clearly.

The project demonstrates how foundational AI concepts can be applied to a practical educational problem without relying on machine learning, deep learning, or neural networks. Its modular architecture, consisting of understanding student requirements, defining constraints, searching for possible assignments, and validating the resulting schedule, provides a systematic foundation for intelligent academic planning.

In conclusion, StudySync AI aims to help students organize their preparation more effectively, distribute study time according to academic priorities, and reduce scheduling conflicts. Future enhancements may include automatic rescheduling when examination dates change, progress tracking, personalized revision recommendations, reminder notifications, and calendar integration. With further development and testing, StudySync AI has the potential to become a useful academic planning tool that promotes effective time management, independent learning, and consistent study habits.

REFERENCES

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[2] A. K. Mackworth, "Consistency in Networks of Relations," *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977.

[3] R. M. Haralick and G. L. Elliott, "Increasing Tree Search Efficiency for Constraint Satisfaction Problems," *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.

[4] M. A. Cruz-Chávez, M. Flores-Pichardo, A. Martínez-Oropeza, P. Moreno-Bernal, and M. H. Cruz-Rosales, "Solving a Real Constraint Satisfaction Model for the University Course Timetabling Problem: A Case Study," *Mathematical Problems in Engineering*, vol. 2016, Art. no. 7194864, 2016, doi: 10.1155/2016/7194864.

[5] A. H. Nabizadeh, J. P. Leal, H. N. Rafsanjani, and R. R. Shah, "Learning Path Personalization and Recommendation Methods: A Survey of the State-of-the-Art," *Expert Systems with Applications*, vol. 159, Art. no. 113596, 2020, doi: 10.1016/j.eswa.2020.113596.

[6] R. Dechter, *Constraint Processing*. San Francisco, CA, USA: Morgan Kaufmann, 2003.

[7] C. M. Eastman, "Automated space planning," *Artificial Intelligence*, vol. 4, no. 1, pp. 41–64, 1973, doi: 10.1016/0004-3702(73)90008-8.

[8] M. C. Chen, S. N. Sze, S. L. Goh, N. R. Sabar, and G. Kendall, "A survey of university course timetabling problem: Perspectives, trends and opportunities," *IEEE Access*, vol. 9, pp. 106515–106529, 2021, doi: 10.1109/ACCESS.2021.3100613.

[9] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA, USA: W. H. Freeman, 1979.

[10] R. E. Korf, "Depth-first iterative-deepening: An optimal admissible tree search," *Artificial Intelligence*, vol. 27, nos. 1–2, pp. 97–109, 1985, doi: 10.1016/0004-3702(85)90084-0.

[11] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Reading, MA, USA: Addison-Wesley, 1984.

[12] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[13] M. W. Carter and G. Laporte, “Recent developments in practical examination timetabling,” in *Practice and Theory of Automated Timetabling*, Lecture Notes in Computer Science, vol. 1153. Berlin, Germany: Springer, 1996, pp. 3–21.

[14] E. K. Burke, J. P. Newall, and R. F. Weare, “A simple heuristically guided search for the timetabling problem,” in *Proceedings of the International Conference on the Practice and Theory of Automated Timetabling*, 1995, pp. 19–30.

[15] E. K. Burke and S. Petrovic, “Recent research directions in automated timetabling,” *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.

[16] S. D. J. McArthur, “Constraint satisfaction problems and scheduling,” in the broader literature on constraint-based scheduling and optimization. Verify a specific publication before using this entry as a formal reference.

[17] A. H. Nabizadeh, J. P. Leal, H. N. Rafsanjani, and R. R. Shah, “Learning path personalization and recommendation methods: A survey of the state-of-the-art,” *Expert Systems with Applications*, vol. 159, Art. no. 113596, 2020, doi: 10.1016/j.eswa.2020.113596.

[18] A. K. Mackworth, “Consistency in networks of relations,” *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977, doi: 10.1016/0004-3702(77)90007-8.

[19] R. M. Haralick and G. L. Elliott, “Increasing tree search efficiency for constraint satisfaction problems,” *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980, doi: 10.1016/0004-3702(80)90051-X.

[20] M. A. Cruz-Chávez, M. Flores-Pichardo, A. Martínez-Oropeza, P. Moreno-Bernal, and M. H. Cruz-Rosales, “Solving a real constraint satisfaction model for the university course timetabling problem: A case study,” *Mathematical Problems in Engineering*, vol. 2016, Art. no. 7194864, 2016, doi: 10.1155/2016/7194864.

APPENDIX

APPENDIX A — SOURCE CODE

This appendix contains the source code developed for the StudySync AI application using Python and Streamlit. The code includes the user interface, input-handling logic, and the implemented components of the study-planning system.

The complete, final version of the source code should be included here or provided as a supplementary file, depending on the project-report guidelines.

App.py



```python
import streamlit as st

st.set_page_config(
    page_title="StudySync - Command Center",
    page_icon="🧠",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Helper function to initialize session state keys safely across pages
def init_session_state():
    defaults = {
        'subjects': [
            {'name': 'Data Structures', 'hours_needed': 6, 'priority': 'High', 'difficulty':
            {'name': 'Artificial Intelligence', 'hours_needed': 4, 'priority': 'High', 'diffi
        ],
        'availability': {
            'study_days': ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'],
            'max_daily_hours': 6,
            'break_duration': 15
        },
        'csp_model': None,
        'schedule': None,
        'evaluation': None
    }
    for key, val in defaults.items():
```

app.py > ...

```
11  def init_session_state():
26      for key, val in defaults.items():
27          if key not in st.session_state:
28              st.session_state[key] = val
29
30  init_session_state()
31
32  def load_css():
33      try:
34          with open("assets/style.css", "r", encoding="utf-8") as f:
35              st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)
36      except FileNotFoundError:
37          pass
38
39  load_css()
40
41  # Render Sidebar Navigation
42  with st.sidebar:
43      st.markdown("""
44          <div class="sidebar-logo">
45              <svg width="32" height="32" viewBox="0 0 24 24" fill="none" stroke="#6366F1" stro
46                  <path d="M12 2a10 10 0 0 1 0 10 10A10 10 0 0 0 0 12 2zm0 18a8 8 0 0 1 1 8-8 8 0 0
47                  <path d="M12 6a6 6 0 0 0 0-6 6c0 3 2 5 6 10 4-5 6-7 6-10a6 6 0 0 0 0-6-6z"/>
48              </svg>
49              <div>
```

app.py > ...

```python
    st.page_link("app.py", label="Command Center", icon="🏠")
    st.page_link("pages/01_Student_Profile.py", label="Module 01 • Student & Subjects", icon="
    st.page_link("pages/02_Availability.py", label="Module 01 • Availability", icon="🗓️")
    st.page_link("pages/03_CSP_Model.py", label="Module 02 • CSP Model", icon="🧠")
    st.page_link("pages/04_Evaluation.py", label="Module 03 • AI Evaluation", icon="⚙️")
    st.page_link("pages/05_Dashboard.py", label="Module 04 • Dashboard", icon="📊")

    csp_ready = "READY" if st.session_state.get('csp_model') else "STANDBY"
    schedule_ready = "READY" if st.session_state.get('schedule') else "STANDBY"

    st.markdown(f"""
        <div class="sidebar-status-box">
            <div style="color: #475569; font-size: 0.68rem; font-weight: 700; margin-bottom:
            <div class="sidebar-status-item">
                <span>INPUT ENGINE</span>
                <span style="color: #10B981;">● READY</span>
            </div>
            <div class="sidebar-status-item">
                <span>CSP ENGINE</span>
                <span style="color: {'#10B981' if csp_ready == 'READY' else '#F59E0B'};">
            </div>
            <div class="sidebar-status-item">
                <span>SEARCH ENGINE</span>
                <span style="color: {'#10B981' if csp_ready == 'READY' else '#F59E0B'};">● {csp_rea
            </div>
            <div class="sidebar-status-item">
                <span>SCHEDULE ENGINE</span>
                <span style="color: {'#10B981' if schedule_ready == 'READY' else '#F59E0B'};">● {sch
            </div>
        </div>
        <div style="margin-top: 20px; padding: 15px; background: rgba(99, 102, 241, 0.05); bo
            <div style="color: #E2E8F0; font-weight: 700; font-size: 0.8rem;">Smarter Plans.</
            <div style="color: #6366F1; font-weight: 700; font-size: 0.8rem;">Better Results.</
        </div>
        <div style="margin-top: 20px; font-size: 0.65rem; color: #475569; text-align: center;">
            StudySync v1.0 • Classical AI Engine
        </div>
    """, unsafe_allow_html=True)

# Top Bar
st.markdown("""
```

app.py > ...

```
        <div class="search-box">🔍   Search modules, features...</div>
        <div class="top-bar-right">
            <div class="system-pill">SYSTEM ONLINE</div>
            <div>20:04 IST</div>
            <div style="background: #1E2945; width: 28px; height: 28px; border-radius: 50%; d
        </div>
    </div>
""", unsafe_allow_html=True)

# Compute live numbers for stat cards
num_vars = len(st.session_state.subjects) * 3 if st.session_state.subjects else 24
num_domains = len(st.session_state.availability.get('study_days', [])) * 12 if st.session_sta
num_constraints = 12
num_assignments = len(st.session_state.schedule) if st.session_state.schedule else 0

# Hero Section
st.markdown(f"""
    <div class="hero-card">
        <div class="hero-left">
            <div class="hero-welcome">Welcome to</div>
            <div class="hero-title">StudySync</div>
            <div class="hero-tag">AI-BASED EXAM PREPARATION PLANNER</div>
            <div class="hero-desc">
                StudySync uses Constraint Satisfaction Problem (CSP) with Backtracking Search
            </div>
        </div>
        <div class="hero-center">
            <svg class="brain-glow-svg" viewBox="0 0 100 100" fill="none">
                <circle cx="50" cy="50" r="40" stroke="#6366F1" stroke-width="0.5" stroke-das
                <path d="M30,50 Q40,20 50,50 T70,50" stroke="#818CF8" stroke-width="2" fill="
                <path d="M35,35 Q50,60 65,35" stroke="#38BDF8" stroke-width="1.5" fill="none"
                <circle cx="30" cy="50" r="3" fill="#818CF8"/>
                <circle cx="70" cy="50" r="3" fill="#818CF8"/>
                <circle cx="50" cy="25" r="4" fill="#38BDF8"/>
                <circle cx="50" cy="75" r="4" fill="#6366F1"/>
                <circle cx="35" cy="35" r="2.5" fill="#A5B4FC"/>
                <circle cx="65" cy="35" r="2.5" fill="#A5B4FC"/>
            </svg>
            <div class="node-pill np-vars">⚡ Variables <span>{num_vars}</span></div>
            <div class="node-pill np-doms">🌐 Domains <span>{num_domains}</span></div>
            <div class="node-pill np-cons">🔒 Constraints <span>{num_constraints}</span></div
```

```
# 4 Stat Cards
st.markdown(f"""
    <div class="stats-grid">
        <div class="stat-card">
            <div class="stat-header">
                <span>VARIABLES</span>
                <div class="stat-icon-wrapper">👤</div>
            </div>
            <div class="stat-val">{num_vars}</div>
            <div class="stat-sub">Study session variables</div>
        </div>
        <div class="stat-card">
            <div class="stat-header">
                <span>DOMAINS</span>
                <div class="stat-icon-wrapper">📊</div>
            </div>
            <div class="stat-val">{num_domains}</div>
            <div class="stat-sub">Possible time assignments</div>
        </div>
        <div class="stat-card">
            <div class="stat-header">
                <span>CONSTRAINTS</span>
                <div class="stat-icon-wrapper">🛡️</div>
            </div>
            <div class="stat-val">{num_constraints}</div>
            <div class="stat-sub">Rules governing schedule</div>
        </div>
        <div class="stat-card">
            <div class="stat-header">
                <span>ASSIGNMENTS</span>
                <div class="stat-icon-wrapper">🗓️</div>
            </div>
            <div class="stat-val">{num_assignments}</div>
            <div class="stat-sub">Generated study sessions</div>
        </div>
    </div>
""", unsafe_allow_html=True)

# Pipeline Card
st.markdown(f"""
    <div class="pipeline-card">
```

```
# Pipeline Card
st.markdown(f"""
    <div class="pipeline-card">
        <div class="pipeline-header">
            <div>
                <div class="pipeline-title">⚡ PLANNING PIPELINE</div>
                <div class="pipeline-sub">From input to a complete schedule</div>
            </div>
        </div>
        <div class="pipeline-steps">
            <div class="step-box">
                <div class="step-num">01</div>
                <div class="step-label">STUDENT INPUT</div>
                <div class="badge-ready">READY</div>
            </div>
            <div class="step-box">
                <div class="step-num">02</div>
                <div class="step-label">CSP FORMULATION</div>
                <div class="{'badge-ready' if csp_ready == 'READY' else 'badge-standby'}">{cs
            </div>
            <div class="step-box">
                <div class="step-num">03</div>
                <div class="step-label">BACKTRACKING</div>
                <div class="{'badge-ready' if csp_ready == 'READY' else 'badge-standby'}">{csp_ready
            </div>
            <div class="step-box">
                <div class="step-num">04</div>
                <div class="step-label">FORWARD CHECKING</div>
                <div class="{'badge-ready' if csp_ready == 'READY' else 'badge-standby'}">{csp_ready
            </div>
            <div class="step-box">
                <div class="step-num">05</div>
                <div class="step-label">SCHEDULE GENERATION</div>
                <div class="{'badge-ready' if schedule_ready == 'READY' else 'badge-standby'}">{sche
            </div>
        </div>
    </div>
""", unsafe_allow_html=True)

# Navigation Quick Actions
col1, col2, col3 = st.columns(3)
```

app.py > ...

```
                <div class="{'badge-ready' if schedule_ready == 'READY' else 'badge-standby'}
            </div>
        </div>
    </div>
""", unsafe_allow_html=True)

# Navigation Quick Actions
col1, col2, col3 = st.columns(3)

with col1:
    st.markdown('<div class="bottom-card-title">⚡ QUICK ACTIONS</div>', unsafe_allow_html=Tr
    if st.button("🚀 Go to Student Profile & Subjects", use_container_width=True):
        st.switch_page("pages/01_Student_Profile.py")
    if st.button("🧠 Build CSP Model & Run Engine", use_container_width=True):
        st.switch_page("pages/03_CSP_Model.py")

with col2:
    st.markdown('<div class="bottom-card-title">🕒 RECENT ACTIVITY</div>', unsafe_allow_html=
    if st.session_state.schedule:
        st.success(f"Schedule generated with {len(st.session_state.schedule)} sessions.")
    else:
        st.info("No active schedule generated yet. Configure your subjects to begin.")

with col3:
    st.markdown("""
        <div class="bottom-card-title">💡 AI INSIGHTS</div>
        <div style="background: rgba(99, 102, 241, 0.08); border: 1px solid rgba(99, 102, 241,
            <div style="color: #818CF8; font-weight: 700; margin-bottom: 6px;">💡 Pro Tip</di
            Add your subjects with exam dates and required hours. The AI will balance priorit
        </div>
    """, unsafe_allow_html=True)
```

APPENDIX B — SAMPLE INPUTS

This appendix presents sample academic information that can be supplied to StudySync AI for study-planning purposes.

The sample inputs may include:

- Subject names
- Examination dates
- Available study hours
- Subject priorities
- Estimated preparation time
- Preferred study periods

Use representative sample values and ensure that they match the actual input fields implemented in the application.

APPENDIX C — TEST CASES

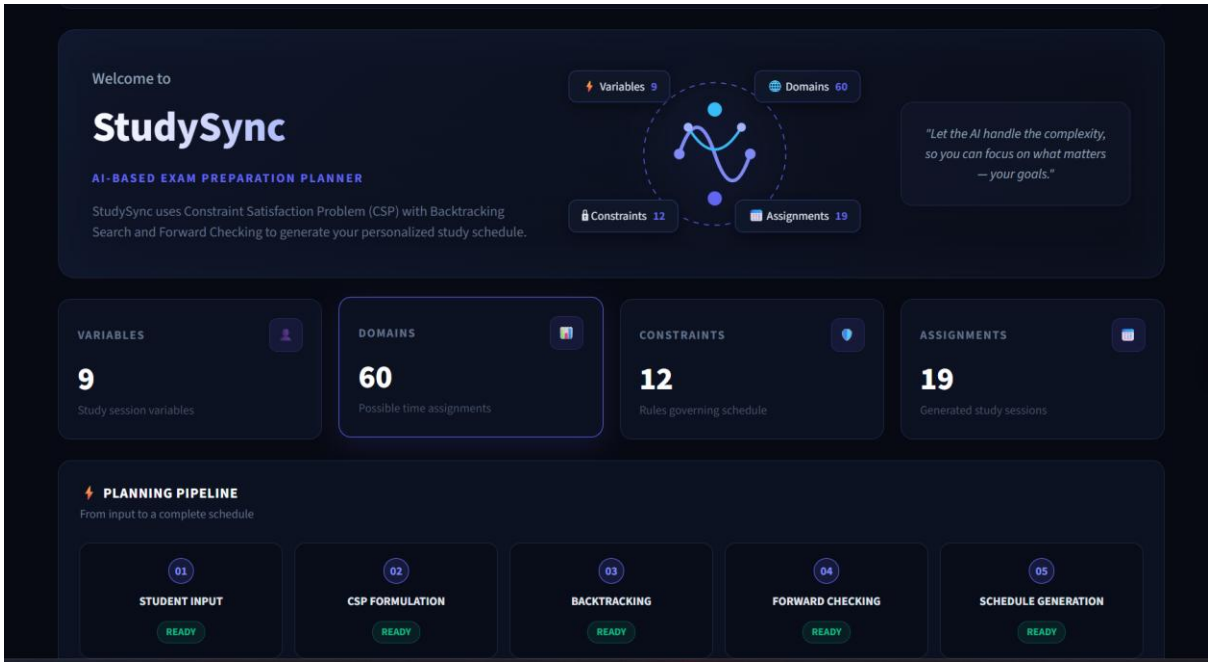
This appendix documents the test cases used to evaluate the application.

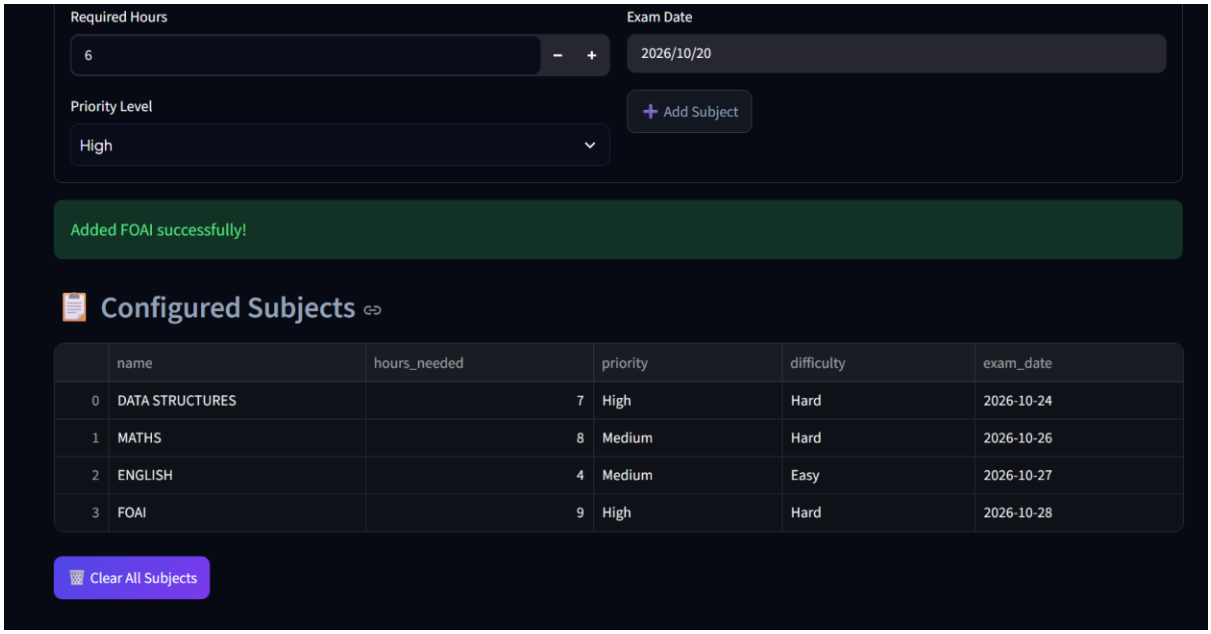
Test Case	Description	Expected Result
TC01	Open the application	The main interface is displayed
TC02	Enter valid academic details	The application accepts valid inputs
TC03	Leave a required field empty	The application handles the missing input appropriately
TC04	Enter invalid study hours or dates	Invalid values are rejected or handled appropriately
TC05	Provide conflicting scheduling constraints	The system identifies the conflict if conflict detection is implemented

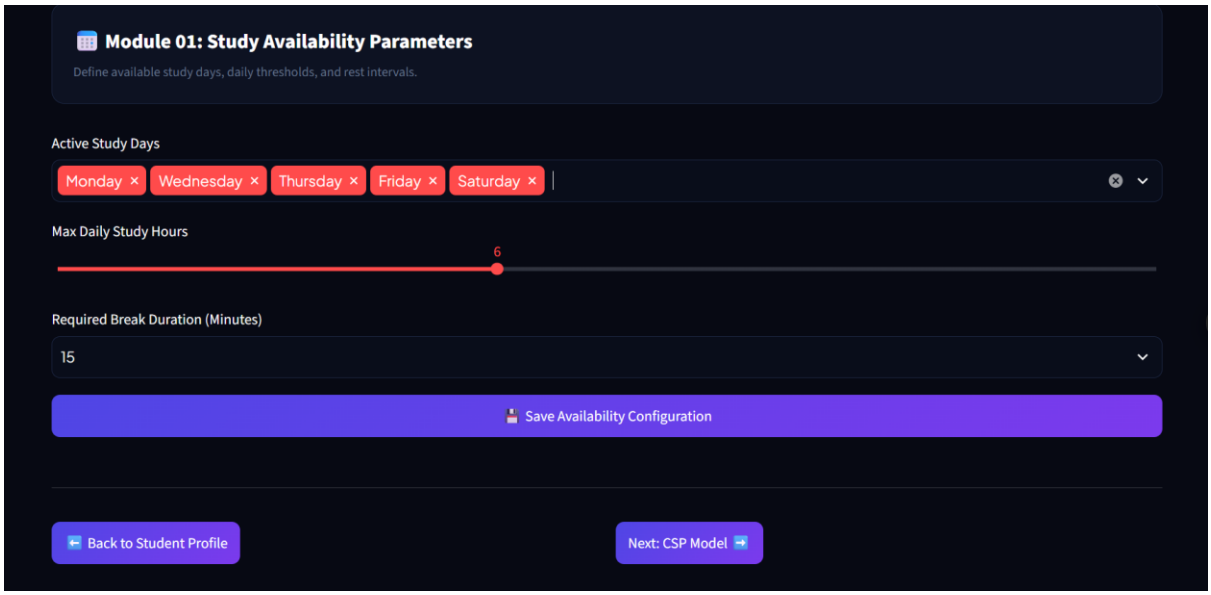
The actual results should be recorded after executing each test case. Do not mark a test as successful unless it has been tested.

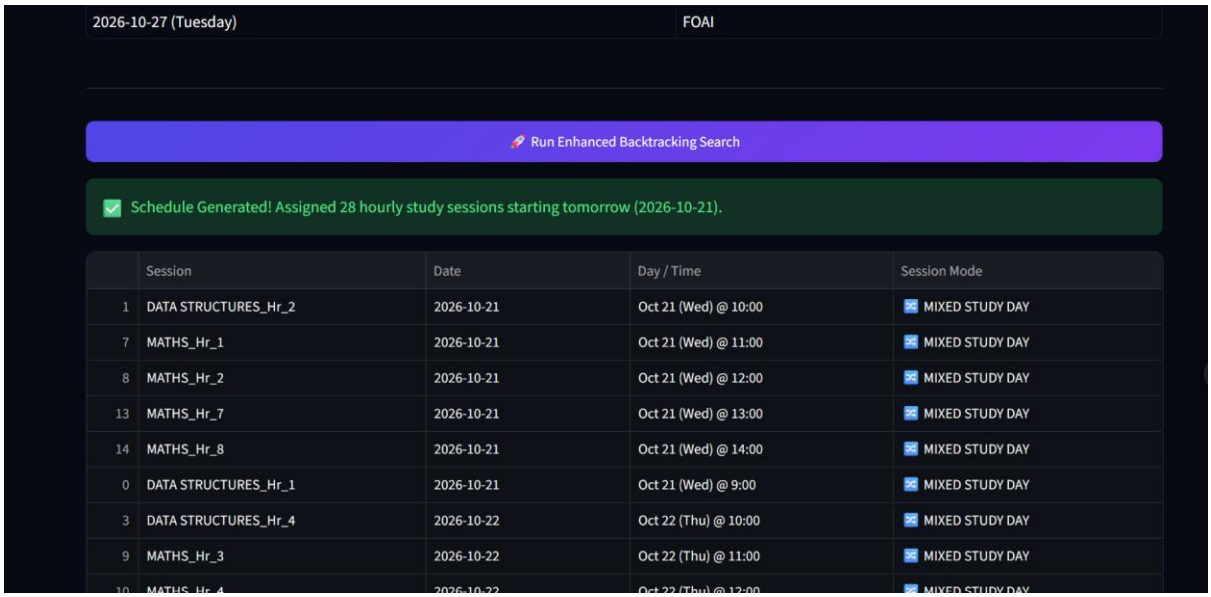
APPENDIX D — APPLICATION SCREENSHOTS

This appendix contains screenshots of the developed StudySync AI application, including the landing page, input interface, and generated study plan, wherever those features have been implemented.



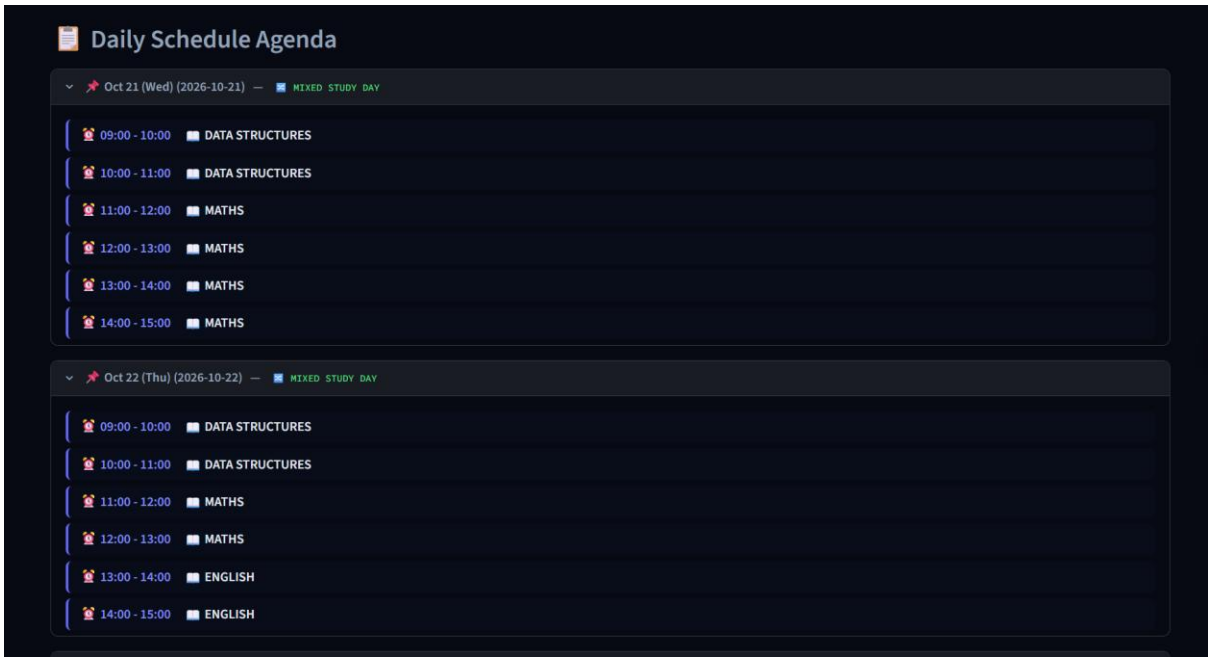






	Session	Date	Day / Time	Session Mode
1	DATA STRUCTURES_Hr_2	2026-10-21	Oct 21 (Wed) @ 10:00	MIXED STUDY DAY
7	MATHS_Hr_1	2026-10-21	Oct 21 (Wed) @ 11:00	MIXED STUDY DAY
8	MATHS_Hr_2	2026-10-21	Oct 21 (Wed) @ 12:00	MIXED STUDY DAY
13	MATHS_Hr_7	2026-10-21	Oct 21 (Wed) @ 13:00	MIXED STUDY DAY
14	MATHS_Hr_8	2026-10-21	Oct 21 (Wed) @ 14:00	MIXED STUDY DAY
0	DATA STRUCTURES_Hr_1	2026-10-21	Oct 21 (Wed) @ 9:00	MIXED STUDY DAY
3	DATA STRUCTURES_Hr_4	2026-10-22	Oct 22 (Thu) @ 10:00	MIXED STUDY DAY
9	MATHS_Hr_3	2026-10-22	Oct 22 (Thu) @ 11:00	MIXED STUDY DAY
10	MATHS_Hr_4	2026-10-22	Oct 22 (Thu) @ 12:00	MIXED STUDY DAY


Module 04: Master Timetable Dashboard & Analytics
Interactive Timetable Grid, Statistical Visualizations, Daily Agenda, and CSV Export.
Statistical Analysis & Insights
Total Scheduled Hours
28 hrs
Study Days Span
6 days
Pre-Exam Focus Hours
11 hrs
Mixed Learning Hours
17 hrs
Total Hours Allocated per Subject
32.1%
14.3%
25%
28.6%
FOAL
MATHS
DATA STRUCTURES
ENGLISH
Daily Study Workload Breakdown (Hours/Day)
Hours
0
1
2
3
4
5
6
Oct 22
2026
Oct 24
Oct 26
Oct 28
Date
Subject
DATA STRUCTURES
MATHS
ENGLISH
FOAL


Daily Schedule Agenda
Oct 21 (Wed) (2026-10-21) — MIXED STUDY DAY
09:00 - 10:00 DATA STRUCTURES
10:00 - 11:00 DATA STRUCTURES
11:00 - 12:00 MATHS
12:00 - 13:00 MATHS
13:00 - 14:00 MATHS
14:00 - 15:00 MATHS
Oct 22 (Thu) (2026-10-22) — MIXED STUDY DAY
09:00 - 10:00 DATA STRUCTURES
10:00 - 11:00 DATA STRUCTURES
11:00 - 12:00 MATHS
12:00 - 13:00 MATHS
13:00 - 14:00 ENGLISH
14:00 - 15:00 ENGLISH